

Complete Guide to Basic Sorting Algorithms (Bubble, Bubble v2, Insertion, Selection)

Sorting is the process of rearranging elements (numbers, strings, objects) into a specific order—usually ascending or descending. In Data Structures, these “basic sorts” matter because they teach core ideas like comparisons, swapping, shifting, and how to analyze **time complexity**.

This guide covers:

- Bubble Sort
 - Bubble Sort v2 (optimized bubble sort)
 - Selection Sort
 - Insertion Sort
with concepts, code, and complexity.
-

1. Bubble Sort (Basic)

Core idea

Bubble sort repeatedly compares **adjacent** elements and swaps them if they are out of order. After one full pass, the **largest** element ends up at the end. Repeating passes gradually moves large values to the right.

How it works

- Pass 1: biggest element “bubbles” to the last position
- Pass 2: second biggest bubbles to second-to-last
- Continue until sorted

C++ Code (Basic Bubble Sort)

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {                // number of passes  
        for (int j = 0; j < n - 1; j++) {            // compare  
neighbors  
            if (arr[j] > arr[j + 1]) {                // swap if out of  
order  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

Time complexity

- Best: $O(n^2)$ (still does all passes)
- Average: $O(n^2)$
- Worst: $O(n^2)$
- Space: $O(1)$

When to use

Mostly for learning. Rarely used in real systems.

2. Bubble Sort v2 (Optimized Bubble Sort)

What changed?

Bubble v2 improves bubble sort with two key ideas:

1. **Stop early** if no swaps happen in a pass (already sorted).

2. **Reduce comparisons** because after each pass, the last part is already sorted.

C++ Code (Bubble v2)

```
void bubbleSortV2(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        for (int j = 0; j < n - 1 - i; j++) { // -i avoids sorted
tail
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }

        if (!swapped) break; // no swaps => already sorted
    }
}
```

Time complexity

- Best: $O(n)$ (already sorted -> one pass, no swaps)
- Average: $O(n^2)$
- Worst: $O(n^2)$
- Space: $O(1)$

Why best case becomes $O(n)$

If the array is already sorted, it does one pass and exits.

3. Selection Sort

Core idea

Selection sort works by repeatedly selecting the **smallest** element from the unsorted portion and swapping it into the correct position.

How it works

- Find the smallest element in the list → swap with index 0
- Find the next smallest → swap with index 1
- Continue until done

C++ Code

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // find index of smallest element in remaining array
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // swap into correct position
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

Time complexity

- Best: $O(n^2)$
- Average: $O(n^2)$

- Worst: $O(n^2)$
- Space: $O(1)$

Key detail

Selection sort does **fewer swaps** than bubble sort, but it still does many comparisons.

4. Insertion Sort

Core idea

Insertion sort builds a sorted section from left to right. For each new element, it inserts it into the correct place in the sorted part—like sorting playing cards in your hand.

How it works

- Assume the first element is sorted
- Take the next element (key)
- Shift larger elements to the right
- Insert key in its correct position

C++ Code

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // shift elements to make space for key
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}
```

```
        // place key
        arr[j + 1] = key;
    }
}
```

Time complexity

- Best: $O(n)$ (already sorted, minimal shifting)
- Average: $O(n^2)$
- Worst: $O(n^2)$ (reverse sorted, lots of shifting)
- Space: $O(1)$

When insertion sort is actually useful

Insertion sort performs well when:

- the data is **already partially sorted**
 - n is small
- It's also used inside more advanced algorithms (like TimSort) for small subarrays.
-

5. Comparing the Algorithms

Conceptual differences

- Bubble sorts by repeated adjacent swaps
- Selection sorts by selecting the minimum each pass
- Insertion sorts by inserting each item into a sorted prefix

Complexity table

Algorithm	Best	Average	Worst	Space	Notes
Bubble (basic)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	simple but inefficient
Bubble v2	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	early exit if sorted
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	few swaps
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	great for nearly-sorted

6. Step-by-step Example (Quick Trace)

Given: [5, 1, 4, 2]

Bubble Sort (one pass idea)

- Compare 5 and 1 → swap → [1, 5, 4, 2]
- Compare 5 and 4 → swap → [1, 4, 5, 2]
- Compare 5 and 2 → swap → [1, 4, 2, 5]
Largest (5) ends at end after pass 1.

Selection Sort (first pass)

- Find min (1) → swap with index 0 → [1, 5, 4, 2]
Then repeat for the remaining portion.

Insertion Sort (inserting)

- Start sorted part: [5]
- Insert 1 into [5] → [1, 5, 4, 2]
- Insert 4 into [1, 5] → [1, 4, 5, 2]

- Insert 2 into $[1, 4, 5] \rightarrow [1, 2, 4, 5]$